



Loop engineering: Designing loops you can actually walk away from by

Hendrik Krack

Loop Engineering

A new term is currently trending across X and capturing the attention of the engineering community: loop engineering.

The core concept is straightforward: move beyond manual interactions with your coding agents. Instead, engineer autonomous loops that handle the execution for you, freeing you up to focus on other tasks, or perhaps design your next loop.

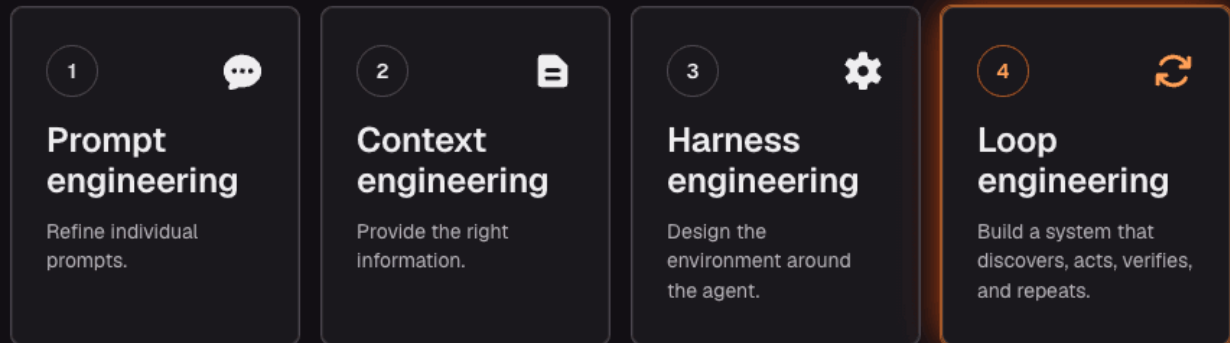
Peter Steinberger (Creator of OpenClaw) kicked off the debate, and Boris (Head of Claude Code) famously quoted "I've moved past manual prompting. Now, I design autonomous cycles that direct Claude and navigate the execution path for me. My real work is engineering the loops themselves."

The industry shifted from prompt engineering in early 2023 to context management, and eventually to harness engineering, which gained traction late in 2025. By the time 2026 was being hailed as "the year of agent harnesses," the focus had already pivoted once more.

Where loop engineering started

The idea isn't even new. Back in January, Geoffrey Huntley was already planting the flag with what he calls the ralph loop - feed an agent the same goal over and over, let it run for hours, and have it find, plan, and resolve the work on its own with nobody in the seat.

HOW THE FIELD EVOLVED



Rather than relying on the model's internal context, progress is preserved through git history, files, and external memory management, which remains the primary technical hurdle. By utilizing a basic bash script, a new agent can seamlessly resume the task exactly where its predecessor stopped once the context window is exhausted.

Anthropic published the same shape from the research side a couple of months earlier: an agent working in shifts, each one showing up with no memory of the last, leaning on notes left on disk to carry on.

What does a loop look like?



While prompt, context, and harness engineering all required you to remain hands-on manually guiding the agent through every turn, loop engineering represents a total shift. It allows you to step away completely by designing a system that operates autonomously, removing the human from the loop entirely.

The fundamental distinction lies in who is in control:

- **Prompts** serve as isolated instructions; the AI provides a single response and then halts, waiting for your next input.
- **Loops** are recursive goals. Once the purpose is established, the system independently navigates the entire circuit, persisting until the objective is fully realized.

What are the paradigms needed for a loop?

Regarding the actual architecture, Addy Osmani provides the most streamlined breakdown: five core building blocks plus a dedicated memory layer. While terminology may vary across different platforms, the underlying functionality remains consistent.

- **Automations:** Something on a schedule that hunts down work and triages it before you've had your coffee.
- **Worktrees:** Parallel agents each have their own checkout, so two of them can't overwrite each other's files.
- **Skills:** Your project's knowledge is written down once: conventions, build steps, the hard-won "don't do it this way" notes, so the agent stops re-guessing every session.
- **Plugins and connectors:** Built on MCP, this is how the loop reaches past the filesystem into the tools you actually use: your issue tracker system, a database, staging environment, Slack.
- **Sub-agents:** The model that wrote the code isn't the one that grades it, because it's a soft grader of its own work.
- **State:** The sixth piece, and the easy one to underrate, memory on disk. A file or a Linear board that tracks what's done and what's next, because the model resets between runs even when the repo doesn't.

How do you design a successful full loop?

Back in February I wired up a loop for a personal project, mostly to see how far it could run without me. It started with real user feedback, as the loop pulled in new requests, triaged them, and drafted a plan for each one worth doing. From there Claude wrote the implementation, CodeRabbit reviewed it in a loop until the diff came back clean, and Claude ran the tests.

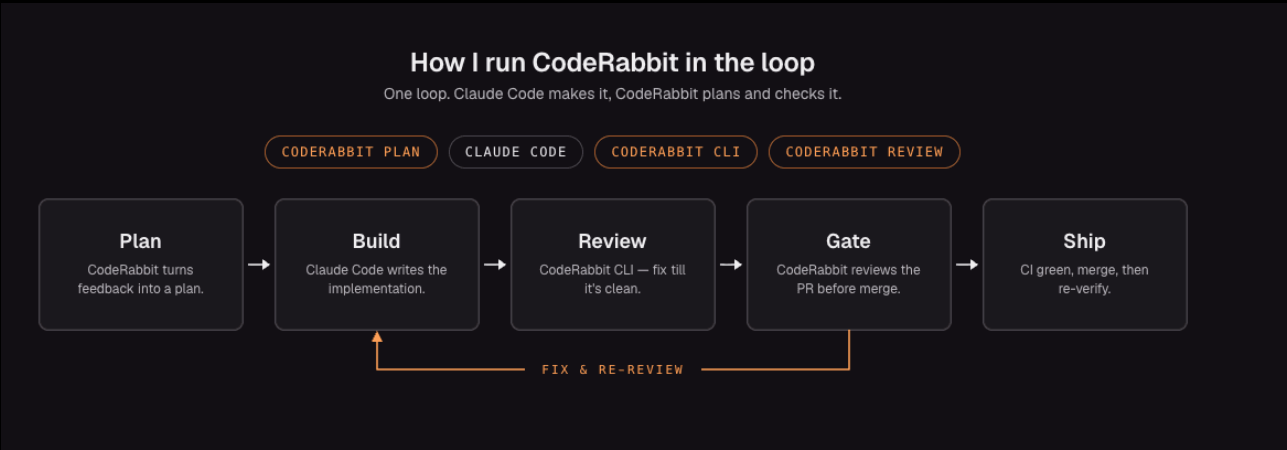
If everything held, the loop waited for CI, merged on its own, and then re-verified the result with a post-merge deploy check. My whole job was deciding what was worth implementing and verifying what the agents produced. I designed the loop once, and it kept shipping.



What actually held it together was boring, and that's the point. The whole thing ran off one scheduled job and a state file: a plain markdown log of what shipped, what failed, and what was still open, so each run resumed instead of starting cold. The project's conventions lived in a skill the agents read every pass, so nobody re-derived the setup from scratch.

And the part that let me leave it alone was the quality gate. Nothing merged unless the tests passed and CodeRabbit's review came back clean, which meant "done" was a signal I could trust, not Claude's opinion of its own work. I kept it on small, checkable changes, the kind a careful junior could ship from a ticket, and left anything that needed real judgment for myself. The loop didn't make those calls, it just made the calls I'd already encoded, over and over, without me.

How CodeRabbit fits into loop engineering?



On the CodeRabbit side it was three pieces working with Claude Code: the planning agent turned raw feedback into a coding plan, the CLI ran reviews right inside the loop so Claude could fix findings before anything became a PR, and the review product was the final gate on the PR itself. Here's the shape of it:

To build or to not build a loop

Before you build one, ask whether the task is even worth it. Loops reward a stable target: rewrite a codebase and the goal holds still, so one verifier (does it build, does the tests pass, does behavior still match) carries every iteration, written once and reused the whole way. But when the conditions keep shifting, the math flips.

If each run needs its own definition of "done," you spend your time rewriting the verifier instead of shipping, and that upkeep eats whatever the loop was supposed to save. Stable goal, build the loop. Moving target, keep it a manual prompt.